Lappeenrannan teknillinen yliopisto
School of Business and Management

Sofware Development Skills

Jasmin Martens, 003347977

LEARNING DIARY, Software Development Skills: Full-Stack MODULE

LEARNING DIARY

Topics: General Information

29.10.2025

I enrolled in the course today and got an overview of the expected learning outcome and assignments that are expected of me. I looked up more about the MERN-Stack and searched for some example projects one could do, such as a chat application, a social media platform, e-commerce website and the like. For now, I still don’t know what I want to do for my submission.

Topics: NodeJs

30.10.2025

I started the tasks with the NodeJs part and began watching the linked video.
Interesting differences between running JavaScript on the browser, vs on node:
- No window object
- No document object
- there is a global object, that has access to setInterval and setTimeout (part of the webAPI).
- process object

So the environment is different, but the JavaScript has the same logic.
When importing ES modules, you have to specify the file type in the import, e.g. “import { function } from “./utils.js”. “./utils” would not work.
When exporting as default, you have to keep exports to one per file. That also means while importing, don’t use curly brackets (since it’s one module only).

I created a Postman account and downloaded the desktop app. Then, I kept following the nodeJS video’s guide on how to create a server with plain nodeJS, up until the file and directory variables. After setting those up I stopped.

Topics: NodeJs

31.10.2025

I continued the NodeJS crash course by following the code examples. In doing so I finished the simple server setup section and all its parts. I found it particularly interesting how simple it is to build a barebone API, when it all comes down to it.

Topics: NodeJs, MongoDB

12.11.2025

I continued the NodeJS crash course by following along with the fs, path, os, url, crypto, events and process. By doing so I finished the NodeJS crash course.
Next I started the MongoDB crash course. Coming from a place of only knowing Databases such as MySQL, the differences explained at the beginning were interesting.
I created an account for MongoDB Atlas and downloaded the shell. I also setup the VsCode Extension for MongoDB.
I followed along until the end of the video.

Topics: MongoDB, ExpressJS

13.11.2025

As Review I setup the mongoDB connection via node without following the video and committed the changes to my repo. (I made sure to store the password in an env variable and not commit the env file).
Afterward I started the express crash course and setup the static folder.

Topics: ExpressJS

14.11.2025

Followed along with the expressJS crash course. I'll try sticking to the module syntax instead of commonJS, but here are some notes for me:

CommonJS is what node modules default to and can be identified by ”const var = require(”var”)”. This one has easier access to __dir and __file, but I do find it easier to stick to the same syntax overall. Because ES6 uses modules, it makes sense to just switch the type in the package.json to ”module” and use the import logic.
If using a middleware, be sure to pass next in your function params.

Topics: ExpressJS, React
15.11.2025

Finished the ExpressJS video tutorial and pushed the updated code to the git repository. I decided to skip following along to the ejs template engines because it’s not as relevant for me right now and I have used a couple of other template engines before (twig, jinja, …).
Afterwards, I found out the linked React Crash Course has received another update, so I am following the 2024 version: https://www.youtube.com/watch?v=LDB4uaJ87e0.
Notes:
- in jsx classes cannot be declared via ”class” but instead className.
- when declaring props, destructuring is useful for having direct access to your params.
- the jsx, react extension on vscode gives useful shortcuts. Typing ”rafce” and hitting enter, gives you the structure for an exported arrow function prop.
- when using an arrow function in jsx like map, don’t use curly brackets as that won’t work. They need to be replaced with parenthesis. (Actually this might have more to do with being inside return, than jsx).

I followed along with the coding project and stopped at 1:29:25 for now.

Topics: React
19.11.2025
I continued with the React Course. Some notes I don’t want to lose:
json-server is a package that can be used to create a mock backend api. When using vite, we can declare a proxy like this in the vite.config:

```
proxy: {
"/api": {
target: "http://localhost:5000",
changeOrigin: true,
rewrite: (path) => path.replace(/^\/api/, ""),
},
},
```

And then use it (example):

```
const apiUrl = isHome ? "/api/jobs?_limit=3" : "/api/jobs"
```

And then we can just store the host in our env file, to keep everything safe and dynamic.

When using useEffect, it’s important to declare an array, even if it’s an empty one, as the function would otherwise loop infinitely.

```
useEffect(() => {}, []);
```

useParams can be used to dynamically fetch the corresponding data from the api:

```
import { useParams } from "react-router-dom";
const JobPage = () => {
const { id } = useParams();
const [job, setJob] = useState(null);
const [loading, setLoading] = useState(true);

useEffect(() => {
const fetchJob = async() => {
try {
const res = await fetch(`/api/jobs/${id}`);
const data = await res.json();
setJob(data);
} catch (error) {
console.log(error);
} finally {
setLoading(false);
}
}
fetchJob();
```

```
}, []);
return loading ? <Spinner /> : (
<h1>{job.title}</h1>
)
}
```

Another way to access data is by creating a dataloader:

```
import { useParams, useLoaderData } from "react-router-dom";

// by creating a dataloader we create a function in this file,
// that can be passed to other components
const JobPage = () => {
const { id } = useParams();
const job = useLoaderData();
return <h1>{job.title}</h1>
}

const jobLoader = async({ params }) => {
const res = await fetch(`/api/jobs/${params.id}`);
const data = await res.json();
return data;
}

export { JobPage as default, jobLoader };
```

and this way we can use it in other components, or in its own component like this:

```
<Route path="/jobs/:id" element={<JobPage/>} loader={jobLoader} />
```

I stopped at the “add job” section, because this was a lot of information to retain.

Topics: React

20.11.2025

I continued with the React Crash Course and learned how to submit forms in React and do post methods to the api. I still need to get used to how functions can be passed down and upstream between components, but overall the way React works makes sense to me so far.

Next, I learned how to edit an existing dataset and have it be updated in the api as well. With that, I finished the React Crash Course.

Topics: MERN

05.12.2025

I started the final project by reviewing the MERN stack along with the MERN playlist. I didn't follow along step by step, as the previous tutorials are more up to date and more in depth. However, there are some new informations I picked up, like using mongoose.

After setting up my first database collection and its routes, I stopped.

Topics: JWT, BCRYPT

06.12.2025

I continued with my project, following along with the tutorial playlist. Authentication is a new topic to me, so for this section I followed along more closely sticking to the tutorial. A couple of important things I want to note down, so I can have them as reference:

```
import { protect } from "../middleware/auth.js";

router.get("/", protect, getGoals);
```

this utilizes a middleware we wrote to authenticate the user. By passing it to the router, we ensure only if the user has the correct token, they will be able to access the route.

```
export const getGoals = async (req, res, next) => {
const goals = await Goal.find({ user: req.user.id });
// get all goals
res.status(200).json(goals);
};
```

Because the auth middleware also passes along the user data, we now have access to user id inside our controller and can easily check for a match.

Later on, I need to create more schemas for different collections that should be in the project but for now I will continue following the tutorial, now heading to the frontend section.

While starting the Redux-Toolkit section I noticed that the structure is significantly different from the current model, so I went to check if there was a more recent redux video in the channel. Only then did I notice that the entire MERN-stack playlist had gotten an update. I should be fine for the previous steps because I was already adjusting the code to more modern approaches as I followed along, but it's good that I noticed it now.
I went back to update a couple of things that were added in the new code.
To not start from scratch I am setting up my frontend with tailwindcss and daisyUI. I haven't used daisyUI before, but it should have most of what I need to set up the layout components without having to do much custom styling. If I wasn't under time pressure I would like to create a custom styling but given the time constraint and deadline approaching, I'll keep custom styling to a minimum.

Topics: Project Work

07.12.2025

I continued with the React part of my project work. First, I implemented the login and registration page. I also added a Navbar, but may change its appearance later. It's useful to have for now to get around though.

Topics: Project Work

08.12.2025

Continuing on, I implemented the goals module by creating a modal that would let a user add new goals and then a component where current goals could be seen. The user has the option to delete or complete their goals.

Topics: Project Work

09.12.2025

I added two new schemas to the backend: movies and movieWatchlist. One will contain a list of movies that have been added by users, and the other will be the connecting piece between movies and users, letting a user add movies they have watched on it.
To get movie information, I registered for "TheMovieDatabase" and implemented their fetchAPI.

Topics: Project Work

10.12.2025

I continued on with the movie implementation and did the frontend functionality of searching for a movie and receiving infos.

Topics: Project Work

14.12.2025

For the last couple of days I continued with the project work but mostly did fixing and a basic UI implementation. I also finished the documentation and some Postman tests. With this, the prototype is finished and ready to be submitted.